

Lecture 18: Data Visualization (Part-1) — Complete Solutions

In-Depth Theoretical Analysis, Code Implementations & Best Practices

This document provides exhaustive, production-grade solutions to all three case-based problems in the **Lecture 18 Practice Problem Set**.

Solution to Case 1: SaaS YoY Revenue Growth & Executive Reporting

1. Perception & Chart Selection Theory

- **Visual Encoding of Temporal Continuity:** Human perception naturally interprets connected lines as continuous temporal progressions (rates of change, velocity, acceleration). When categorical bar charts are used for time-series data, discrete vertical bars imply isolated, independent events rather than an evolving trajectory.
- **Anscombe's Quartet & Data-Ink Ratio Formulation:** Tabular summaries (e.g., printed spreadsheets) fail to convey inflection points. Two datasets can share identical mean and variance yet display drastically divergent trends. A multi-series line plot maximizes Edward Tufte's **Data-Ink Ratio** η :

$$\eta = \frac{\mathcal{I}_{\text{data}}}{\mathcal{I}_{\text{total}}} = 1.0 - \frac{\mathcal{I}_{\text{non-data}}}{\mathcal{I}_{\text{total}}}$$

Where:

- $\mathcal{I}_{\text{data}}$: Integral of non-redundant visual elements dedicated strictly to displaying data information (trendlines, data points, markers).
- $\mathcal{I}_{\text{total}}$: Total ink area of the visual display (including background, borders, ticks, and grids).
- Domain: $\eta \in (0, 1]$, where optimal graphical efficiency is achieved as $\eta \rightarrow 1.0$. It minimizes cognitive friction and enables executive decision-makers to immediately

spot the growth divergence between FY 2023 and FY 2024.

2. Complete Python Implementation

```
import matplotlib.pyplot as plt

# Monthly Financial Datasets
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
rev_2023 = [12, 14, 15, 18, 19, 22, 24, 25, 28, 30, 31, 35]
rev_2024 = [15, 18, 22, 25, 29, 34, 38, 42, 45, 50, 54, 60]

# 1. Establish high-resolution figure canvas (12 x 6 inches)
plt.figure(figsize=(12, 6), dpi=300)

# 2. Multi-dataset line plots utilizing fast format strings ('fmt')
# 'go--' -> green color ('g'), circle marker ('o'), dashed line ('--')
# 'bs-' -> blue color ('b'), square marker ('s'), solid line ('-')
plt.plot(months, rev_2023, 'go--', linewidth=2, markersize=7, label='FY 2023 Revenue')
plt.plot(months, rev_2024, 'bs-', linewidth=2.5, markersize=8, label='FY 2024 Revenue')

# 3. Informative axis titling and hierarchy
plt.title('SaaS Monthly Recurring Revenue (MRR) YoY Comparison', fontsize=16, fontweight='bold')
plt.xlabel('Fiscal Month', fontsize=12, fontweight='semibold')
plt.ylabel('Revenue (in Lakhs ₹)', fontsize=12, fontweight='semibold')

# 4. Subtle grid and executive legend styling
plt.grid(True, linestyle=':', alpha=0.6, color='gray')
plt.legend(loc='upper left', frameon=True, shadow=True, facecolor='#f8f9fa', fontweight='bold')

# 5. Production export avoiding label clipping
plt.savefig('saas_revenue_report.png', dpi=300, bbox_inches='tight', transparent=False)
plt.show()
```

3. Production Export Analysis

- **Why default exports fail:** By default, `plt.savefig()` uses the screen DPI (often 72–100 DPI), resulting in raster pixelation on 4K projectors or print media. Additionally, outside bounding boxes (`bbox`) are unadjusted, meaning long tick labels or titles often get clipped at figure boundaries.
- **The Solution:** Specifying `dpi=300` ensures crisp print-grade rendering, while `bbox_inches='tight'` computes the tightest bounding box encompassing all Artist elements (titles, axis labels, legends, tick marks) before rasterization.

Solution to Case 2: Enterprise Financial Audit & Spending Variance

1. Mathematical Coordinate Offset Derivation

Let N denote the total number of categorical entities (departments), and let $K = 2$ denote the number of comparative series (Series₁: Budget, Series₂: Spend).

Definition 1 (Baseline Categorical Domain):

The discrete baseline index of the i -th category along the category axis is given by:

$$x_i = i, \quad \forall i \in \{0, 1, 2, \dots, N - 1\}$$

Condition 1 (Non-Overlapping Criterion):

Let w denote the uniform bar width. To guarantee zero collision between adjacent departmental clusters, w must satisfy the inequality:

$$K \cdot w < 1.0 \implies w < \frac{1}{K} = \frac{1}{2} = 0.5 \quad (\text{Chosen: } w = 0.38)$$

System of Shifted Coordinate Equations:

The shifted position coordinates $(y_{1,i}, y_{2,i})$ for category i are determined by the symmetric piecewise system:

$$\begin{cases} y_{1,i} = x_i - \frac{w}{2} & (\text{Series 1: Budget}) \\ y_{2,i} = x_i + \frac{w}{2} & (\text{Series 2: Spend}) \end{cases}$$

Center Tick Alignment Theorem:

To ensure the categorical tick mark t_i lies precisely equidistant between the two comparative bars:

$$t_i = \frac{y_{1,i} + y_{2,i}}{2} = \frac{\left(x_i - \frac{w}{2}\right) + \left(x_i + \frac{w}{2}\right)}{2} = x_i$$

Thus, category labels are rendered directly at the unshifted coordinate x_i , guaranteeing perfect optical symmetry.

2. Complete Python Implementation (Horizontal Grouped Bar Chart)

```
import matplotlib.pyplot as plt
import numpy as np

departments = ['R&D / Innovation', 'Brand Marketing', 'Global Sales', 'Human Resources', 'Product Development']
budget = [45, 30, 60, 15, 40]
spend = [48, 26, 68, 14, 47]

# 1. Coordinate Offsets
y_indices = np.arange(len(departments))
bar_height = 0.38

fig, ax = plt.subplots(figsize=(11, 6), dpi=150)

# 2. Render horizontal bars
bars_budget = ax.barh(y_indices - bar_height/2, budget, height=bar_height, color='#2b5c8f')
bars_spend = ax.barh(y_indices + bar_height/2, spend, height=bar_height, color='#d95f02')

# 3. Direct Labeling using modern ax.bar_label() container method
ax.bar_label(bars_budget, fmt='₹%dCr', padding=4, fontsize=9, color='#2b5c8f', fontweight='bold')
ax.bar_label(bars_spend, fmt='₹%dCr', padding=4, fontsize=9, color='#d95f02', fontweight='bold')

# 4. Axis ticks and Top-to-Bottom priority inversion
ax.set_yticks(y_indices)
ax.set_yticklabels(departments, fontsize=11, fontweight='semibold')
ax.invert_yaxis() # Puts R&D (first department) at the top

# 5. Decoration
ax.set_xlabel('Expenditure (in Crores ₹)', fontsize=12, fontweight='bold')
ax.set_title('Departmental Budget Allocation vs. Actual Expenditure Audit', fontsize=12, fontweight='bold')
ax.legend(loc='lower right', frameon=True, shadow=True)
ax.grid(axis='x', linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```

3. Comparison: `plt.text()` vs `ax.bar_label()`

- **Classic `plt.text()` Loop:** Requires iterating manually over each bar:

```
for rect in bars:
    width = rect.get_width()
    ax.text(width + offset, rect.get_y() + rect.get_height()/2, f'₹{width}Cr
```

Prone to manual offset errors and tedious formatting logic.

- **Modern `ax.bar_label()` (Matplotlib ≥ 3.4):** Takes the `BarContainer` directly, handles automated placement, accepts string formatters (`fmt='₹%dCr'`), and guarantees zero label distortion.

Solution to Case 3: Venture Capital Portfolio & Sector Allocation

1. 4D Visual Encoding & Alpha Transparency

In multivariate visualization, a 2D Cartesian scatter canvas is formally extended to represent 4-dimensional observations:

$$\mathcal{P}_i = (X_i, Y_i, S_i, C_i) \in \mathbb{R}^4$$

Where each attribute is mathematically encoded via the channel mapping:

$$\left\{ \begin{array}{ll} X_i \in \mathbb{R}^+ & \text{(Total Funding Raised in \$M)} \\ Y_i \in \mathbb{R} & \text{(Annual Revenue Growth Rate in \%)} \\ S_i = \kappa \cdot h_i & \text{(Marker Area Scaling, Headcount } h_i, \kappa = 2.5) \\ C_i = \phi(v_i) & \text{(Colormap Mapping: } v_i \in [1, 10] \xrightarrow{\text{viridis}} \mathbf{c}_i \in [0, 1]^3) \end{array} \right.$$

Optical Density & Cluster Overplotting via Alpha (α):

When m distinct observation points overlap at identical coordinates, the transmitted background luminance I through m overlapping circular markers is governed by the discrete Beer-Lambert extinction model:

$$I_{\text{transmitted}} = I_0 \cdot (1 - \alpha)^m$$

Where:

- I_0 : Initial background intensity.
- $\alpha \in (0, 1]$: Matplotlib alpha transparency coefficient (`alpha=0.75`).
- m : Local cluster density of overlapping points.

As density m increases, local luminance decreases exponentially, visually revealing dense clusters to human perception.

2. Complete Python Implementation (4D Scatter + Arrow Annotation)

```
import matplotlib.pyplot as plt
import numpy as np

# Deterministic Seed
np.random.seed(42)
n_startups = 50
funding = np.random.uniform(2, 60, n_startups)
growth = funding * 5 + np.random.normal(0, 30, n_startups)
team_size = np.random.uniform(10, 200, n_startups)
valuation_tier = np.random.uniform(1, 10, n_startups)

# Injected Outlier (Apex AI)
funding = np.append(funding, [50])
growth = np.append(growth, [420])
team_size = np.append(team_size, [250])
valuation_tier = np.append(valuation_tier, [10])

plt.figure(figsize=(11, 6), dpi=150)

# 4D Scatter Execution
scatter = plt.scatter(
    funding, growth,
    s=team_size * 2.5,
    c=valuation_tier,
    cmap='viridis',
    alpha=0.75,
    edgecolors='black',
    linewidth=1.2
)

# Colorbar Legend
cbar = plt.colorbar(scatter)
cbar.set_label('Valuation Tier Scale (1-10)', fontsize=11, fontweight='semibold')
```

```
# Pointer Annotation with Arrowprops
plt.annotate(
    'Apex AI (Star Unicorn)\nFunding: $50M | Growth: 420%\nTeam: 250 Employees',
    xy=(50, 420),          # Point to annotate
    xytext=(20, 380),      # Text location
    arrowprops=dict(facecolor='crimson', shrink=0.08, width=2, headwidth=8),
    bbox=dict(boxstyle='round,pad=0.5', facecolor='#fff3cd', edgecolor='crimson',
              fontsize=10, fontweight='bold'
    )

plt.title('VC AI Portfolio: 4D Performance Matrix', fontsize=14, fontweight='bold')
plt.xlabel('Total Funding Raised ($ Millions)', fontsize=12, fontweight='semibold')
plt.ylabel('Annual Revenue Growth Rate (%)', fontsize=12, fontweight='semibold')
plt.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```

3. Capital Allocation: Donut Chart Implementation

- **Why Donut Charts Outperform Standard Pie Charts:** Human cognitive perception assesses **linear lengths and bar heights** far more accurately than **angles and slice areas**. Pie charts crowd labels around slices and make comparisons between similar angles nearly impossible. A Donut Chart replaces the ambiguous center vertex with a hollow core, guiding the reader's eye along the **arc length** of each wedge.

```
sectors = ['LLMs & Generative AI', 'Robotics & Automation', 'Computer Vision', 'Fi
allocation = [40, 20, 15, 15, 10]
colors = ['#4e79a7', '#f28e2b', '#e15759', '#76b7b2', '#59a14f']
explode = [0.08, 0, 0, 0, 0] # Emphasize primary sector

plt.figure(figsize=(7, 7), dpi=150)

wedges, texts, autotexts = plt.pie(
    allocation,
    labels=sectors,
    autopct='%1.1f%%',
    startangle=140,
    explode=explode,
    colors=colors,
    pctdistance=0.78,
    wedgeprops=dict(width=0.4, edgecolor='white', linewidth=2) # Hollow hole creat
)

for autotext in autotexts:
    autotext.set_fontsize(10)
    autotext.set_fontweight('bold')
```

```
plt.title('Portfolio Capital Allocation by AI Sector', fontsize=14, fontweight='bold')  
plt.tight_layout()  
plt.show()
```

